

Implementazione di QoS in reti SDN

tramite differenziazione del traffico e rate limiting con OpenFlow

Di Costanzo Mariagrazia DE9000158 Pucino Valentina DE9000144

1. Introduzione e Obiettivo

Il progetto affronta il tema “Implementazione di regole su switch in SDN” applicato alla gestione della Quality of Service. L’obiettivo è dimostrare come un controller SDN, tramite protocollo OpenFlow, possa riconoscere dinamicamente il tipo di traffico attraversante la rete e applicare politiche di trattamento differenziate.

Il caso d’uso scelto è la distinzione tra traffico “voce”, simulato con UDP – sensibile a jitter e perdita di pacchetti – e traffico “bulk”, simulato con TCP – tollerante ai ritardi, ma invasivo in termini di banda occupata.

2. Architettura e scelte progettuali

L’ambiente vede Mininet 2.3.0 per l’emulazione della rete, Ryu 4.34 come controller SDN, Open vSwitch con supporto OpenFlow1.3.

La topologia è definita da un singolo switch OVS collegato a tre host:

```
from mininet.topo import Topo

class QoSTopo(Topo):
    #metodo per costruire la rete
    def build(self):
        s1 = self.addSwitch('s1')

        h1 = self.addHost('h1') #sorgente UDP
        h2 = self.addHost('h2') #sorgente TCP
        h3 = self.addHost('h3') # destinazione comune

        #aggiungo i link
        self.addLink(h1,s1, bw = 5)
        self.addLink(h2,s1, bw = 5)
        self.addLink(h3,s1, bw = 5)

topos = { 'qostopo': (lambda:QoSTopo()) }
```

```
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet> exit
```

Figura 1: topologia della rete

La scelta di una topologia a singolo switch è motivata dalla necessità di concentrare la contesa di banda in un unico punto definito (la porta in uscita verso h3), rendendo l’effetto della QoS osservabile e misurabile. Ai link è stata assegnata una banda limitata per riprodurre una condizione di congestione realistica.

Meccanismo di base: il controller installa una regola di table-miss (priority 0) che inoltra al controller ogni pacchetto privo di corrispondenza in tabella. Alla ricezione di un Packet-In, il controller esamina l’header IP del pacchetto per determinarne il protocollo e installare una flow rule specifica per tipo di traffico e direzione – tramite il match su in_port, necessario per evitare che una stessa regola catturi erroneamente anche il traffico di ritorno in connessioni bidirezionali come TCP.

```

class QoS_Switch(app_manager.RyuApp):
    #dichiariamo la versione di OpenFlow
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(QoS_Switch, self).__init__(*args, **kwargs)
        self.mac_to_port = {} # dizionario usato per il forwarding

    #funzione chiamata automaticamente quando uno switch si collega al controller
    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        datapath = ev.msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        self.logger.info("Switch connesso: %s", datapath.id) #log connection

        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER, ofproto.OFPCML_NO_BUFFER)]

        self.add_flow(datapath, 0, match, actions)

    def add_flow(self, datapath, priority, match, actions):
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser

        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS, actions)]

        mod = parser.OFPFlowMod(datapath = datapath, priority = priority, match=match, instructions = inst)

        datapath.send_msg(mod)

    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def packet_in_handler(self, ev):
        msg = ev.msg
        self.logger.info("Packet-In ricevuto!")
        datapath = msg.datapath
        ofproto = datapath.ofproto
        parser = datapath.ofproto_parser
        in_port = msg.match['in_port'] #porta fisica da cui arriva il pacchetto

        pkt = packet.Packet(msg.data)
        eth = pkt.get_protocol(ethernet.ethernet) #estrazione header ethernet per leggere indirizzi MAC

        dst = eth.dst
        src = eth.src
        dpid = datapath.id
        self.mac_to_port.setdefault(dpid, {})

        self.mac_to_port[dpid][src] = in_port

        if dst in self.mac_to_port[dpid]:
            out_port = self.mac_to_port[dpid][dst] # se conosciamo il destinatario
        else:
            out_port = ofproto.OFPP_FLOOD #altrimenti: flooding

        actions = [parser.OFPActionOutput(out_port)]

```

Figura ii: qos_controller.py

3. Evoluzione tecnica:

Una prima versione del controller assegna priority differenziate alle flow rule: 100 per il traffico UDP, 10 per il traffico TCP.

```

if out_port != ofproto.OFPP_FLOOD:
    ip_pkt = pkt.get_protocol(ipv4.ipv4)

    if ip_pkt is not None:
        if ip_pkt.proto == 17:
            priority = 100
            match = parser.OFPMatch(eth_type = 0x0800, ip_proto = 17, in_port = in_port)
            self.add_flow(datapath, priority, match, actions)
            self.logger.info("Regola UDP installata!")
        elif ip_pkt.proto == 6:
            priority = 10
            match = parser.OFPMatch(eth_type = 0x0800, ip_proto = 6, in_port = in_port)
            self.add_flow(datapath, priority, match, actions)
            self.logger.info("Regola TCP installata!")

    out = parser.OFPPacketOut(
        datapath = datapath, buffer_id = msg.buffer_id, in_port = in_port, actions = actions, data = msg.data)
    datapath.send_msg(out)

```

Figura iii: qos_controller.py

È stato condotto un test comparativo, generando traffico UDP e TCP simultaneo in due condizioni: senza QoS (con priority uguali) e con QoS (con priority differenziate). Il confronto ha mostrato che la sola priority non produce una differenza significativa su jitter e perdita di pacchetti del traffico UDP.

```

mininet> h3 iperf -s -u -p 5001 > /tmp/udp_server.txt &
mininet> h3 iperf -s -p 5002 > /tmp/tcp_server.txt &

```

Figura iv: generazione traffico

```

so@Ubuntu-S0:~$ cat /tmp/udp_server.txt
-----
Server listening on UDP port 5001
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.3 port 5001 connected with 10.0.0.1 port 36985
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-15.0201 sec  3.75 MBytes  2.10 Mbits/sec  2.265 ms  0/2678 (0%)
[ 1] 0.0000-15.0201 sec  1 datagrams received out-of-order
so@Ubuntu-S0:~$ cat /tmp/tcp_server.txt
-----
Server listening on TCP port 5002
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.3 port 5002 connected with 10.0.0.2 port 56742
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-15.4689 sec  5.50 MBytes  2.98 Mbits/sec
so@Ubuntu-S0:~$ cat /tmp/udp_client.txt
-----
Client connecting to 10.0.0.3, UDP port 5001
Sending 1470 byte datagrams, IPG target: 5607.60 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 36985 connected with 10.0.0.3 port 5001
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-15.0061 sec  3.75 MBytes  2.10 Mbits/sec
[ 1] Sent 2679 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-15.0201 sec  3.75 MBytes  2.10 Mbits/sec  2.264 ms  0/2678 (0%)
[ 1] 0.0000-15.0201 sec  1 datagrams received out-of-order
so@Ubuntu-S0:~$ cat /tmp/tcp_client.txt
-----
Client connecting to 10.0.0.3, TCP port 5002
TCP window size: 102 KByte (default)
-----
[ 1] local 10.0.0.2 port 56742 connected with 10.0.0.3 port 5002
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-15.4790 sec  5.50 MBytes  2.98 Mbits/sec

```

Figura v: Test con QoS

```

so@Ubuntu-S0:~$ cat /tmp/udp_server.txt
-----
Server listening on UDP port 5001
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.3 port 5001 connected with 10.0.0.1 port 46805
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-15.0193 sec  3.75 MBytes  2.10 Mbits/sec  2.666 ms  0/2678 (0%)
[ 1] 0.0000-15.0193 sec  1 datagrams received out-of-order
so@Ubuntu-S0:~$ cat /tmp/tcp_server.txt
-----
Server listening on TCP port 5002
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.3 port 5002 connected with 10.0.0.2 port 58888
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-15.4958 sec  5.63 MBytes  3.05 Mbits/sec
so@Ubuntu-S0:~$ cat /tmp/udp_client.txt
-----
Client connecting to 10.0.0.3, UDP port 5001
Sending 1470 byte datagrams, IPG target: 5607.60 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 46805 connected with 10.0.0.3 port 5001
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-15.0064 sec  3.75 MBytes  2.10 Mbits/sec
[ 1] Sent 2679 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer    Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-15.0193 sec  3.75 MBytes  2.10 Mbits/sec  2.665 ms  0/2678 (0%)
[ 1] 0.0000-15.0193 sec  1 datagrams received out-of-order
so@Ubuntu-S0:~$ cat /tmp/tcp_client.txt
-----
Client connecting to 10.0.0.3, TCP port 5002
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 58888 connected with 10.0.0.3 port 5002
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-15.4924 sec  5.63 MBytes  3.05 Mbits/sec

```

Figura vii: Test senza QoS

La priority di una flow rule determina quale regola si applica a un pacchetto in caso di sovrapposizione tra più match, ma non influisce sul trattamento del pacchetto una volta instradato – tutti i pacchetti destinati alla stessa porta di uscita condividono la medesima coda fisica, indipendentemente dalla priority della regola che li ha instradati.

Per ottenere un controllo effettivo sul piano dati, è stato introdotto un Meter OpenFlow, applicato esclusivamente al traffico TCP, con un limite di banda di 2000 kbps e azione di scarto dei pacchetti in eccesso. Ogni flow rule TCP referencia il Meter tramite un’istruzione dedicata, applicata prima dell’istruzione di inoltrare.

```

#funzione chiamata automaticamente quando uno switch si collega al controller
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    self.logger.info("Switch connesso: %s", datapath.id) #log connection

    match = parser.OFPMatch()
    actions = [parser.OFPAActionOutput(ofproto.OFPP_CONTROLLER, ofproto.OFPCML_NO_BUFFER)]

    self.add_flow(datapath, 0, match, actions)

    bands = [parser.OFPMeterBandDrop(rate = 2000, burst_size = 0)] #scarta i pacchetti oltre la soglia con nessuna tolleranza
    req = parser.OFPMeterMod( datapath = datapath, command = ofproto.OFPMC_ADD, flags = ofproto.OFPMF_KBPS, meter_id = 1, bands = bands)
    datapath.send_msg(req)
    self.logger.info("Meter TCP creato(limite 2000 kbps)")

```

Figura vii: modifica funzione con Meter

```

elif ip_pkt.proto == 6:
    priority = 10
    match = parser.OFPMatch(eth_type = 0x0800, ip_proto = 6, in_port = in_port)
    inst = [parser.OFPInstructionMeter(1, ofproto.OFPII_METER), parser.OFPInstructionActions(ofproto.OFPII_APPLY_ACTIONS, actions)]
    mod = parser.OFPFlowMod(datapath = datapath, priority = priority, match = match, instructions = inst)
    datapath.send_msg(mod)
    self.logger.info("Regola TCP installata con Meter!")

```

Figura viii: qos_controller_meter.py

```

so@Ubuntu-S0:~$ cat /tmp/udp_server.txt
-----
Server listening on UDP port 5001
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.3 port 5001 connected with 10.0.0.1 port 50204
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-15.0187 sec  3.75 MBytes  2.10 Mbits/sec  1.826 ms  0/2678 (0%)
[ 1] 0.0000-15.0187 sec  1 datagrams received out-of-order
so@Ubuntu-S0:~$ cat /tmp/tcp_server.txt
-----
Server listening on TCP port 5002
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.3 port 5002 connected with 10.0.0.2 port 35680
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-16.2611 sec  3.88 MBytes  2.00 Mbits/sec
so@Ubuntu-S0:~$ cat /tmp/udp_client.txt
-----
Client connecting to 10.0.0.3, UDP port 5001
Sending 1470 byte datagrams, IPG target: 5607.60 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 50204 connected with 10.0.0.3 port 5001
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-15.0062 sec  3.75 MBytes  2.10 Mbits/sec
[ 1] Sent 2679 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-15.0187 sec  3.75 MBytes  2.10 Mbits/sec  1.826 ms  0/2678 (0%)
[ 1] 0.0000-15.0187 sec  1 datagrams received out-of-order
so@Ubuntu-S0:~$ cat /tmp/tcp_client.txt
-----
Client connecting to 10.0.0.3, TCP port 5002
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.2 port 35680 connected with 10.0.0.3 port 5002
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-16.2866 sec  3.88 MBytes  2.00 Mbits/sec

```

Figura viiix: Test con Meter

4. Risultati sperimentali

I risultati confermano che solo l'introduzione del Meter produce un effetto misurabile: il traffico TCP viene effettivamente contenuto entro il limite impostato, lasciando banda residua stabile per il traffico UDP, il cui jitter risulta il più basso tra le tre configurazioni testate.

```

so@Ubuntu-S0:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
[sudo] password di so:
cookie=0x0, duration=97.220s, table=0, n_packets=2680, n_bytes=4052160, priority=100,udp,in_port="s1-eth1" actions=output:"s1-eth3"
cookie=0x0, duration=95.597s, table=0, n_packets=1998, n_bytes=5899096, priority=10,tcp,in_port="s1-eth2" actions=output:"s1-eth3"
cookie=0x0, duration=95.591s, table=0, n_packets=1773, n_bytes=117046, priority=10,tcp,in_port="s1-eth3" actions=output:"s1-eth2"
cookie=0x0, duration=82.182s, table=0, n_packets=2, n_bytes=340, priority=100,udp,in_port="s1-eth3" actions=output:"s1-eth1"
cookie=0x0, duration=108.263s, table=0, n_packets=28, n_bytes=3770, priority=0 actions=CONTROLLER:65535
so@Ubuntu-S0:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
cookie=0x0, duration=46.071s, table=0, n_packets=2680, n_bytes=4052160, priority=50,udp,in_port="s1-eth1" actions=output:"s1-eth3"
cookie=0x0, duration=44.025s, table=0, n_packets=2120, n_bytes=6038220, priority=50,tcp,in_port="s1-eth2" actions=output:"s1-eth3"
cookie=0x0, duration=44.007s, table=0, n_packets=1874, n_bytes=123712, priority=50,tcp,in_port="s1-eth3" actions=output:"s1-eth2"
cookie=0x0, duration=31.025s, table=0, n_packets=2, n_bytes=340, priority=50,udp,in_port="s1-eth3" actions=output:"s1-eth1"
cookie=0x0, duration=55.145s, table=0, n_packets=19, n_bytes=3068, priority=0 actions=CONTROLLER:65535
so@Ubuntu-S0:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
cookie=0x0, duration=74.333s, table=0, n_packets=2678, n_bytes=4049136, priority=100,udp,in_port="s1-eth1" actions=output:"s1-eth3"
cookie=0x0, duration=71.975s, table=0, n_packets=1831, n_bytes=4591026, priority=10,tcp,in_port="s1-eth2" actions=meter:1,output:"s1-eth3"
cookie=0x0, duration=71.970s, table=0, n_packets=1488, n_bytes=100456, priority=10,tcp,in_port="s1-eth3" actions=meter:1,output:"s1-eth2"
cookie=0x0, duration=59.302s, table=0, n_packets=1, n_bytes=170, priority=100,udp,in_port="s1-eth3" actions=output:"s1-eth1"
cookie=0x0, duration=87.177s, table=0, n_packets=22, n_bytes=2994, priority=0 actions=CONTROLLER:65535

```

Figura ix: flow table

5. Conclusioni

Il progetto ha permesso di implementare e confrontare due meccanismi di QoS in ambiente SDN: la differenziazione tramite priority delle flow rule e la rate limiting tramite Meter OpenFlow. L'esperimento ha evidenziato un limite – la priority da sola agisce solo a livello di matching, non sul piano dati – e ha mostrato come i Meter costituiscano uno strumento corretto per il controllo di banda realmente efficace.